Audio Dispatcher for Unity DOTS

Audio Dispatcher is a production-ready, high-performance bridge between Unity's Data-Oriented Technology Stack (DOTS/ECS) and the classic Managed `AudioSource` system.

Triggering standard Unity audio from pure ECS is notoriously difficult because managed objects cannot be accessed inside Burst-compiled jobs. This package solves that problem using a lock-free `NativeQueue` architecture, Double Buffering, and a highly optimized Zero-GC object pool.

✨ Key Features

- ⚡ **100% Burst Compatible:** Trigger sounds directly from `IJobEntity` or `ISystem` without performance penalties.
- 🚀 **Zero Main Thread Stalls:** Uses a Double Buffering Command Architecture. The main thread never waits for your ECS workers.
- 🗑️ **Zero GC Allocations:** Pre-allocated object pools for both One-Shot and Looping sounds.
- 🧠 **Smart Voice Stealing:** If the audio pool is full, the system calculates priorities and automatically replaces the quietest/furthest sound.
- 🔗 **Shadow Tracking (Loops):** Attach looping sounds (like engine noises) to moving entities. The sound follows the entity and stops automatically when the entity is destroyed, without modifying your core archetypes.
- 💎 **Fluent API:** Clean, readable, and chainable syntax for triggering sounds.
- 🛠️ **Stable Hash IDs:** Auto-generates C# `const int` IDs for your audio clips using stable hashes. Reordering your audio database will never break your code.

📦 Installation

Install via Unity Package Manager (Git URL)

1. Open your Unity project.
2. Go to **Window** > **Package Manager**.
3. Click the + button in the top-left corner and select **Add package from git URL...**
4. Paste the following URL and click **Add**:

```
https://github.com/sniveler-code/com.snivelercode.audio-dispatcher.git
```

Install via manifest.json

Alternatively, open Packages/manifest.json and add the following line to your "dependencies" block:

```
"com.snivelercode.audio-dispatcher": "https://github.com/sniveler-code/com.snivelercode.audio-dispatcher.git"
```

🚀 Quick Start

1. Create an Audio Database

Right-click in your Project view and select `Create > SnivelerCode > Audio Database`. Add your AudioClips to the list, tweak volumes, and assign AudioMixer groups.

2. Generate C# IDs

Select your new Audio Database asset. In the Inspector, click the green `Generate C# Constants` button. This will generate a file with stable Hash IDs for your sounds.

3. Scene Setup

Create an empty `GameObject` in your sub-scene. Add the AudioSettingsAuthoring component to it and assign your Audio Database asset.

4. Play Sounds from Burst Jobs (Fluent API)

You can now trigger sounds from anywhere in your ECS code using the elegant Fluent API:

```
using SnivelerCode.AudioDispatcher.Runtime;
using Unity.Burst;
using Unity.Entities;

[BurstCompile]
public partial struct MyCombatSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        // 1. Get the Audio Writer (Read-Only access allows parallel job scheduling)
        var audioSingleton = SystemAPI.GetSingleton<NativeAudioSystem.Singleton>();

        state.Dependency = new CombatJob
        {
            AudioWriter = audioSingleton.Writer
        }.ScheduleParallel(state.Dependency);
    }
}
```

```
[BurstCompile]
public partial struct CombatJob : IJobEntity
{
    public NativeQueue<AudioEvent>.ParallelWriter AudioWriter;
    private void Execute(in LocalTransform transform)
    {
        // 2. Trigger a One-Shot sound using the Fluent API
        AudioIDs.EXPLOSION.Shot(transform.Position)
          .Volume(0.8f)
          .Pitch(1.1f)
          .Apply(AudioWriter);
    }
}
```

5. Play Looping Sounds

To play a looping sound that follows an entity (e.g., a car engine):

```
AudioIDs.ENGINE_LOOP.Loop(entity)
    .Volume(0.5f)
    .Apply(AudioWriter);
```

The system will automatically track the entity's position and stop the sound when the entity is destroyed.

🏗️ Under the Hood (Architecture)

This package was built with strict Data-Oriented Design principles:

- **Cache-Line Alignment:** The `AudioEvent` struct uses `[StructLayout(LayoutKind.Explicit)]` to overlap Entity and `float3` Position data. This compresses the struct to exactly 32 bytes, allowing exactly two events to fit perfectly into a standard 64-byte L1 CPU cache line.
- **Double Buffering:** ECS Jobs write to `Queue A` while the Main Thread reads from `Queue B`. In the next frame, they swap. This completely eliminates `Dependency.Complete()` stalls.
- **Unmanaged Brain:** All pooling logic, priority calculations, and lifetime tracking are done inside a Burst-compiled `IJob`. The Main Thread acts only as a "dumb executor" that applies a flat array of commands to the Unity C++ API.

🎮 Samples

The package includes a complete **Tank Battle Demo** (`Samples~/DemoScene`). It demonstrates chaotic projectile collisions, moving tank engine loops, global entity destruction, and spatial audio routing. Import it via the Package Manager to see the system in action!